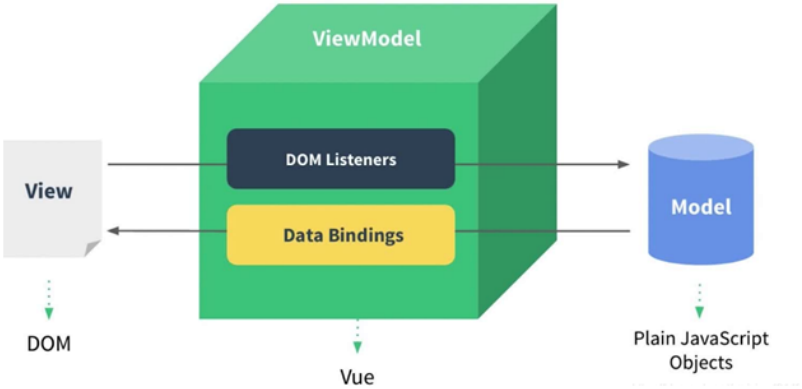
下图不仅概括了MVVM模式（Model-View-ViewModel），还描述了在Vue.js中ViewModel是如何和View以及Model进行交互的。



什么是渐进式框架？

Vue的核心库和一些插件：视图层渲染、组件、路由(vue-router)、状态管理（vuex）

vue2与Vue3的响应式原理

vue2的响应式原理：es5中 Object.defineProperty

vue3的响应式原理：es6中 proxy

<!--

组件的data必须是函数，并且带return的

组件是可复用的，组件是有一个具有单独的自身功能模块，这个模块只属于自己的HTML模板

data是函数可以避免数据污染。

-->

---

vue2

```
// 创建Vue实例
    var vm=new Vue({ //viewModel 管理者
        // el 属性指向的是View ,可以绑定id和类
        el:"#app", //elem 元素
        data:{// 提供数据 Model(数据)
            msg:"hello Vues"
        }
```

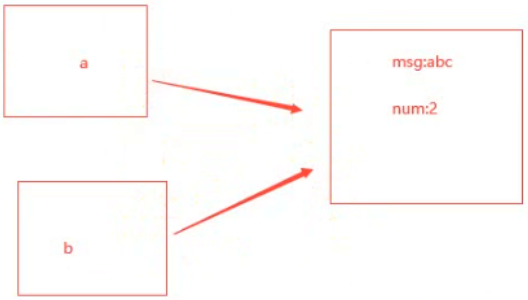
```
})
console.log(vm)
// 两种方式 el和在实例对象后.$mount("id")挂载元素
```

---

vue3

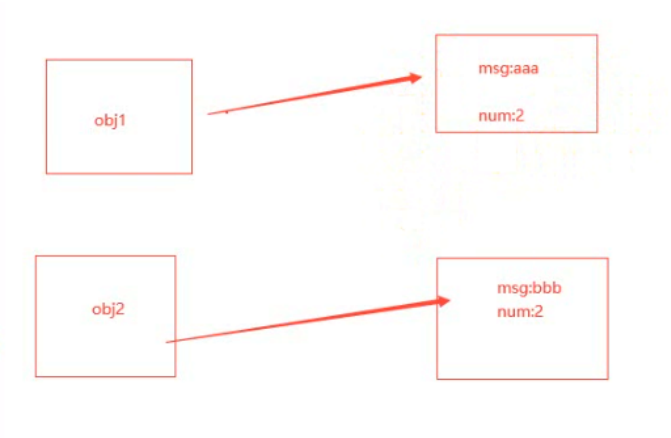
```
// var vm =new Vue({}) vue3不支持，直接创建Vue实例
// new Vue({  }).$mount("#app")
//将这个对象添加到组件实例中
var vm=Vue.createApp({ //选项
    data(){
        return{
            msg2:"hello Vue3"
        }
    }
})


var app=vm.mount("#app");//挂载到根组件上（元素）
console.log(vm)//Object
console.log(app)//Proxy
```

会互相影响




```
let msg={
    msg:'aa',
    num:1
}
function data(){
    return msg
}
let obj1=data()
let obj2=data()
//修改obj1的属性msg为"bb"，那么obj2的属性msg也会被改为"bb"
```

不会互相影响




```
function data1(){
        return{
            msg:'aa',
            num:1
        }
    }
    let obj3=data1()
    let obj4=data1()
    //修改obj3的属性msg为"abc"，那么obj4的属性msg不会被改为"abc"
```